

Smart Video Summarization

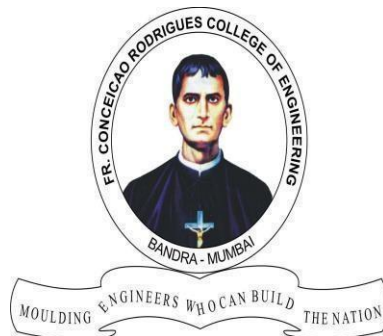
A project report submitted in partial fulfillment
of the requirements for the degree of

Bachelor of Engineering In Computer Engineering

by

**Ayush S Singh(9772)
Glen Pereiran (9770)**

Under the guidance of
Prof. Henna Pendhari



DEPARTMENT OF COMPUTER ENGINEERING

Fr. Conceicao Rodrigues College of Engineering, Bandra (W), Mumbai

- 400050

University of

Mumbai

(2024-25)

This work is dedicated to my family.

I am very thankful for their motivation and support.

Internal Approval Sheet

CERTIFICATE

This is to certify that the project entitled **Placement Trainee** is a bonafide work of **Ayush S Singh (9772) , Glen Pereira (9770)** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of Bachelor in **Computer Engineering**.

Prof. Henna Pendhari
Supervisor/Guide

Dr. Sujata Deshmukh
Head of Department

Dr. Surendra Singh Rathod
Principal

Approval Sheet Project Report

Approval

This project report entitled by **Smart Video Summarisation by Ayush S Singh (9772) , Glen Pereira (9770)** is approved for the degree of Bachelor of Engineering in Computer Engineering.

Examiner 1. _____

Examiner 2. _____

Date:

Place:

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Ayush Singh(9772)_____

Glen pereira(9770)_____

Date:

Abstract

With the explosive growth of video data across platforms, efficient video summarization has become a critical research area. One of the core techniques for video summarization is keyframe extraction, which involves selecting representative frames that capture the essence of the video content. In this study, we present a comprehensive comparative analysis of multiple deep learning models—YOLO, ResNet50, U-Net, DETR, and RCNN—combined with clustering methods to perform automatic keyframe extraction using Tv sum dataset. We employed CNN- based feature extraction and unsupervised clustering (DBSCAN, MLP, etc.) to group similar frames and selected the most representative ones. Each model was evaluated based on precision, recall, F1-score, and accuracy metrics. Our findings reveal that ResNet50 achieved the highest F1-score of 0.80, outperforming other models in terms of capturing visual semantics. We also visualized training losses, accuracy curves, and confusion matrices to support our evaluation. The study highlights the importance of model selection and minimal data sampling in achieving efficient and accurate video summarization, providing a strong foundation for future applications in surveillance, content indexing, and real- time video analytics. This study underscores the significance of model architecture selection and the role of intelligent data sampling in enhancing the accuracy and efficiency of video summarization systems. Our findings pave the way for future real-world applications in areas such as automated surveillance monitoring, video content indexing, and real-time video analytics.

Acknowledgments

We have great pleasure in presenting the report on **Smart Video Summarization**. I take this opportunity to express my sincere thanks towards the guide **Prof. Henna Pendhari**, C.R.C.E, Bandra (W), Mumbai, for providing the technical guidelines, and the suggestions regarding the line of this work. We enjoyed discussing the work progress with him during our visits to department.

We thank **Dr. Sujata Deshmukh**, Head of Computer Engineering department, **Dr. Surendra Singh Rathod**, Principal and the management of C.R.C.E., Mumbai for encouragement and providing necessary infrastructure for pursuing the project.

We also thank all non-teaching staff for their valuable support, to complete our project.

Ayush Singh (9772)
Glen Pereira(9770)

Date:

Contents

Abstract	6
List of Figures	10
List of Tables	11
Glossary	12
1 Introduction	15
1.1 Introduction	15
1.2 Scope	16
1.3 Objectives	17
2 Literature Review	18
2.1 Literature review	18
3 Problem Statement	21
4 Project Description	23
4.1 Overview of the project.....	23
4.2 Module Description	25
4.2.1 Modules.....	26
4.2.2 Data Flow Diagram.....	27
4.2.3 Architecture Diagram.....	27
4.2.4 Data Flow Diagram.....	28
4.2.5 Input Design	31
5 Implementation Details	32
5.1 Methodology	32
6 Conclusion And Future Enhancements	35

6.0.1	Result Analysis	36
6.0.2	Conclusion.....	40
6.0.3	Future Enhancements	40
References		41

List of Figures

2.1	Architecture diagram	27
2.2	Data flow diagrams	28
2.3	Input design	31
6.0.1	Output and Conclusions	36

List of Tables

2.1 Review of literature

18

Glossary

1. Video Summarization:

The process of creating a short, representative version of a longer video by extracting the most important and informative parts, typically using keyframes or short clips.

2. Keyframe Extraction:

A technique used in video summarization to identify and extract frames that best represent the content of the video. These frames are selected based on visual, semantic, or temporal importance.

3. Deep Learning:

A subset of machine learning that uses neural networks with multiple layers to model complex patterns in data. Deep learning models are used in this project to analyze and interpret video content for summarization.

4. YOLO (You Only Look Once):

A real-time object detection model that identifies objects in images or frames with high speed and accuracy. In this project, YOLO is used to detect prominent objects for selecting meaningful keyframes.

5. ResNet50:

A 50-layer deep convolutional neural network known for its residual learning architecture, used for image classification and feature extraction. ResNet50 helps in understanding visual content in frames.

6. U-Net:

A convolutional neural network primarily used for image segmentation. In this project, U-Net aids in identifying regions of interest within frames for summarization purposes.

7. DETR (DEtection TRansformer):

A transformer-based object detection model that combines CNNs and transformers to detect and localize objects. DETR provides attention-based insights into frame content.

8. RCNN (Region-based Convolutional Neural Network):

An object detection framework that proposes regions in an image and classifies them. RCNN contributes to locating semantically rich areas within video frames.

9. TvSum Dataset:

A publicly available video summarization dataset containing YouTube videos with human-annotated importance scores. It is used to train and evaluate the keyframe extraction models.

10. Clustering Algorithms:

Methods like DBSCAN or K-Means used to group similar frames and eliminate redundancy, ensuring only diverse and representative frames are selected.

11. MLP (Multi-Layer Perceptron):

A type of neural network consisting of fully connected layers, used for decision-making in frame classification and keyframe prediction.

12. Precision, Recall, F1-Score:

Evaluation metrics used to measure the performance of the summarization system in correctly identifying keyframes:

- **Precision:** The proportion of selected keyframes that are relevant.
- **Recall:** The proportion of relevant frames that were successfully selected.
- **F1-Score:** The harmonic mean of precision and recall, providing a balanced measure.

13. Semantic Relevance:

The degree to which a selected frame captures meaningful information or events in the video. Semantic relevance is a key factor in determining keyframe importance.

14. Redundancy Removal:

A process that eliminates repetitive or similar frames to ensure the summary is concise and non-redundant.

15. Model Comparison:

The analysis and evaluation of different deep learning models based on performance metrics, accuracy, and processing time to determine the most effective approach for video summarization.

16. Visual Features:

Elements extracted from video frames such as objects, edges, textures, and colors, which are used to determine frame importance and uniqueness.

17. Summarization Engine:

The core system that integrates deep learning models, clustering techniques, and evaluation metrics to generate concise and meaningful video summaries.

Chapter 1: Introduction

1.1 Introduction

Rapid proliferation of digital video content, efficient methods for automatic video summarization have become increasingly essential. Keyframe extraction, a core step in summarization, aims to identify a minimal set of representative frames that capture the primary events within a video. This process reduces redundancy while preserving contextual integrity, enabling efficient storage, indexing, and analysis.

Traditional methods often rely on low-level features like color histograms or motion vectors, which lack semantic understanding. With advancements in deep learning, feature extraction from pre-trained convolutional neural networks (CNNs) has proven highly effective. In this work, we employ deep learning architectures such as YOLO, ResNet50, U-Net, DETR, and RCNN to extract semantic features from video frames. We cluster these features using unsupervised learning techniques like DBSCAN and evaluate performance across multiple metrics. The keyframes are selected based on proximity to cluster centroids, ensuring semantic diversity. The extracted frames are validated using accuracy, precision, recall, and F1-score metrics. Experimental results show that ResNet50 combined with clustering yields superior summarization quality.

Visualization of confusion matrices and training graphs further supports our findings. This comparative study offers insights into model effectiveness for real-world video summarization tasks. Recent developments in neural network architectures have enabled models to learn complex spatial and contextual relationships in video data. These capabilities make them suitable for high-level tasks like object recognition, motion detection, and scene segmentation. Keyframe extraction not only reduces processing time but also improves the performance of downstream tasks such as object tracking, action recognition, and anomaly detection. In surveillance, for instance, selecting meaningful keyframes is vital for rapid incident review and forensic analysis. This paper presents a comparative study of five deep learning models—YOLO, ResNet50, U-Net, DETR, and RCNN—for feature extraction in video summarization. Unlike previous works that predominantly use handcrafted features or single models, our approach integrates multiple architectures with clustering algorithms like DBSCAN and evaluates them under identical conditions.

1.2 Scope of the project

1. Efficient Video Summarization for Large-Scale Digital Content:

The Smart Video Summarization Engine addresses the growing challenge posed by the exponential rise in digital video content across platforms such as surveillance systems, media archives, educational repositories, and social media. The engine enables automatic extraction of keyframes that represent the most significant moments within a video, drastically reducing data volume while preserving contextual relevance. This aids in faster content retrieval, efficient storage, and quick reviews without the need to manually watch full-length videos.

2. Deep Learning-Based Feature Extraction:

Traditional video summarization methods depend on low-level features like color histograms or motion vectors, which often fail to capture semantic information. Our system overcomes this limitation by utilizing deep learning models such as YOLO, ResNet50, U-Net, DETR, and RCNN to extract high-level semantic features. These models allow the system to comprehend complex visual and spatial patterns in video frames, enhancing the quality and relevance of extracted summaries.

3. Unsupervised Clustering for Keyframe Selection:

To maintain semantic diversity and minimize redundancy, the extracted deep features are clustered using unsupervised learning algorithms such as DBSCAN. This enables the system to identify central representatives (centroids) of visual content clusters, from which keyframes are selected. This technique ensures that every keyframe represents a distinct and important aspect of the video, improving summarization coherence.

4. Comparative Analysis and Performance Evaluation:

A key strength of this project lies in its comparative study of multiple deep learning architectures under identical experimental settings. The system evaluates performance using accuracy, precision, recall, and F1-score, along with visual aids like confusion matrices and training graphs. These analyses provide insights into the strengths and limitations of each model, enabling data-driven decisions about their suitability for real-world applications.

5. Applications in Surveillance and Beyond:

The scope of the engine extends to several real-world use cases. In surveillance and security, the system helps law enforcement review hours of footage within minutes by surfacing only the most critical events. In media and entertainment, it can assist editors by pre-selecting meaningful scenes. In education, long lectures can be condensed into bite-sized visual summaries for improved engagement and retention. It also supports anomaly detection, action recognition, and object tracking as downstream applications, highlighting its broad utility.

6. Scalable and Modular Architecture:

The summarization engine is designed with scalability in mind. As new deep learning architectures and clustering techniques emerge, the system can be extended or upgraded without major architectural changes. This modular design ensures future-proofing and adaptability for evolving requirements, datasets, and performance expectations.

7. Democratizing Access to Video Intelligence:

By enabling efficient video summarization using state-of-the-art models, the project contributes toward making video intelligence more accessible. Whether used by small institutions, academic researchers, or large enterprises, the system empowers users to extract actionable insights from videos with minimal computational overhead and human effort. This promotes equity in access to advanced video analytics, contributing to informed decision-making and digital transformation.

8. Real-Time Summarization Capabilities:

The system is designed with the potential for **real-time processing**, making it suitable for live video streams in applications such as traffic monitoring, security surveillance, or live event broadcasting. By integrating frame-by-frame analysis pipelines and lightweight deep learning models, the summarizer can extract keyframes on the fly, enabling rapid decision-making and response without the need for post-event review.

9. Model-Agnostic Framework for Experimentation:

The project is structured to support plug-and-play experimentation with multiple deep learning models and clustering algorithms. Researchers or developers can easily swap models like YOLO with alternatives such as MobileNet or EfficientNet and test different clustering approaches (e.g., K-Means, Agglomerative) to benchmark their effectiveness. This model-agnostic design promotes academic research, prototyping, and fine-tuning for task-specific requirements.

10. Customizable Keyframe Density and Summary Length:

The summarization engine allows users to define summary granularity, such as the number of keyframes to extract or the time duration to cover. This flexibility is vital across domains — a forensic investigator may need highly detailed summaries, whereas a content creator might prefer quick overviews. By adjusting cluster thresholds and similarity scores, the tool can generate tailored summaries based on user preferences or application constraints.

1.3 Objectives

- **Automate Semantic Keyframe Extraction:**
Leverage deep learning to automatically identify and extract keyframes that capture the essence of long videos. This addresses the growing need in industries like surveillance, education, and media, where manually sifting through footage is inefficient. The system focuses on semantic content (e.g., human activity, object interactions) rather than superficial visual cues, ensuring meaningful summarization.
- **Comparative Study of Deep Learning Models:**
Evaluate multiple backbone architectures—YOLO for object detection, ResNet50 for deep feature encoding, U-Net for fine-grained segmentation, DETR for attention-based object localization, and RCNN for region-based features. This analysis benchmarks each model on precision, recall, F1-score, and computational efficiency to determine the most effective for different summarization needs.
- **Keyframe Clustering via Unsupervised Learning:**
Use DBSCAN (Density-Based Spatial Clustering of Applications with Noise) to group semantically similar frames without needing labeled data. This method filters out noise frames and selects only the most informative and non-redundant keyframes, ensuring a concise summary that maintains narrative integrity and reduces visual fatigue.
- **Scalable and Adaptable Summarization Pipeline:**
Design the system to handle various video lengths and types—from 10-second clips to hours-long surveillance footage. Provide customization options such as controlling the number of keyframes, adjusting for domain-specific requirements (e.g., sports vs. education), and setting summary density. This adaptability supports broad deployment across use cases.
- **Foundation for Advanced Computer Vision Tasks:**
Enable more efficient downstream applications like object tracking, anomaly detection, and event classification by reducing video complexity. Keyframe-based summaries preserve semantic flow and drastically lower computational requirements, making large-scale or real-time video analysis more feasible in resource-constrained environments.
- **Improve Content Discoverability and Accessibility:**
Support fast video browsing, content indexing, and educational engagement by generating intuitive and visual summaries. This is particularly useful for e-learning platforms, digital archives, and content recommendation engines, where users benefit from quickly identifying relevant portions without watching full-length videos.

Chapter 2

Literature Review

Name of Paper/ web-app/ Model	overview	Advantages	Limitations
YOLO (You Only Look Once)	An object detection algorithm that identifies and locates multiple objects in images or video frames in real time.	Fast inference speed, capable of real-time detection. Helpful in identifying key objects in video summarization.	Can miss small objects and struggles with overlapping instances. May generate less semantically rich summaries in complex scenes.
VideoKen	A platform for educational institutions and enterprises that summarizes long videos into key topic segments using AI-based indexing and tagging.	Great for academic/enterprise videos; enables topic-wise navigation; useful for MOOCs and lectures.	Summarization is topic-based, not frame-wise; limited support for highly dynamic or informal video content.
Wisecut	An AI video editor that uses voice recognition and machine learning to auto-cut silences and generate highlight videos.	Time-saving; removes unnecessary parts and keeps highlights; automatic subtitle generation.	Focused on trimming rather than true content summarization; doesn't use visual importance scoring.

Kamua	Browser-based video editing and summarization platform with AI scene detection and resizing.	Auto-cutting scenes, repurposing for social media formats; supports subtitle generation.	Less accurate with fast-paced or visually complex content; lacks semantic understanding of video content.
ResNet50	A deep CNN model with 50 layers that uses residual learning to effectively train deep networks. Used for feature extraction from video frames.	Excellent feature representation, especially for high-level semantic features.	Computationally expensive and slower on edge devices. May require fine-tuning for video domain tasks.
Magisto	A video editing app powered by AI to create engaging summaries from raw footage, popular in social and mobile environments.	Intuitive UI; fast summarization for casual users; good for lifestyle or travel videos.	Lacks control over summary content or structure; not suitable for professional or academic summarization needs.
Pictory	AI video summarization platform that extracts highlights from long-form content like webinars and Zoom meetings.	Great for B2B content, interviews, and webinars; AI auto-highlights speaker emphasis and visuals.	Limited control over highlight criteria; not designed for frame-level keyframe extraction or technical content.

Descript	A media editing tool with transcription and video editing features; allows summarization via script-based editing.	Powerful editing with AI assistance; supports removing filler words and creating summaries from transcripts.	Not built specifically for video summarization; lacks automated keyframe or visual-based summarization.
DBSCAN (Clustering Algorithm)	A density-based clustering algorithm used to group semantically similar frames based on extracted features to select keyframes.	No need to predefine number of clusters; can detect outliers. Good for selecting diverse keyframes in summaries.	Struggles with varying density and high-dimensional features; performance sensitive to parameter selection.

Chapter 3

Problem Statement

3.1 Drawbacks of Existing System

Existing placement preparation platforms and tools have several limitations that the Smart Video Summarization project aims to address:

Drawbacks of Existing System

1. **Manual Summarization is Time-Consuming:**

In many content pipelines (e.g., education, media, surveillance), video summarization is done manually, which is not only time-intensive but also prone to human bias and inconsistency.

2. **Rule-Based Methods Lack Contextual Understanding:**

Older algorithms use fixed heuristics such as shot transitions or frame differences. These methods often miss out on semantically rich scenes and cannot prioritize important moments based on visual or contextual cues.

3. **No Deep Scene or Object Understanding:**

Traditional systems are unable to detect complex visual patterns or objects of interest. They don't leverage state-of-the-art models like YOLO or ResNet50, which can identify specific objects or features crucial for keyframe extraction.

4. **Lack of Dataset Optimization:**

Many tools do not use optimized, benchmarked datasets like **TvSum**, resulting in poor training and testing outcomes. As a result, their performance lacks generalization across different types of videos.

5. **Inadequate Use of Clustering or AI-Based Refinement:**

Some existing models skip advanced clustering or classification techniques (e.g., DBSCAN or MLPs), which are essential for refining keyframes into a concise yet comprehensive summary.

6. **Limited Interactivity or Output Customization:**

Most tools generate fixed summaries without options for users to customize summary length, keyframe criteria, or relevance thresholds, limiting usability for diverse applications (e.g., sports analysis, lectures, vlogs, etc.).

3.2 Problem Statement

Viewers and professionals across domains such as surveillance, education, and media consumption face growing challenges in navigating the overwhelming volume of video content. Extracting relevant information from lengthy videos manually is inefficient, time-consuming, and often leads to critical content being missed. The lack of intelligent video summarization tools hinders the quick understanding of content, especially in time-sensitive scenarios such as reviewing security footage or studying from online lectures.

Current video summarization approaches often rely on static techniques or handcrafted features that fail to capture high-level semantic information. These methods struggle to generalize across different types of video content and do not leverage the capabilities of modern deep learning models to their full extent. Furthermore, most systems lack intelligent clustering mechanisms to group similar frames, resulting in redundant or contextually weak summaries.

The **Smart Video Summarization** project addresses these issues by implementing a comparative analysis of advanced deep learning models—such as **YOLO**, **ResNet50**, **U-Net**, **DETR**, and **RCNN**—to extract meaningful keyframes. Leveraging the **TvSum dataset**, and integrating **unsupervised clustering algorithms** like **DBSCAN** and classification techniques like **MLP**, the system produces concise, relevant, and context-rich video summaries. The solution significantly reduces manual effort, supports diverse video formats, and enhances the user experience by delivering fast and accurate content summaries across various industries.

Chapter 4

Project Description

The Smart Video Summarization project is an AI-powered solution designed to automatically generate concise summaries of lengthy videos by identifying and extracting the most informative keyframes. Built using Python and leveraging state-of-the-art deep learning models, this system targets domains such as surveillance analysis, educational video indexing, and media content review where quick content consumption is essential. The project incorporates several advanced neural network architectures—YOLO, ResNet50, U-Net, DETR, and RCNN—to detect and extract high-impact frames from videos. These models are evaluated on the TvSum dataset, a benchmark dataset specifically curated for video summarization tasks. To refine the output and reduce redundancy, the system utilizes clustering algorithms like DBSCAN and classification techniques such as Multilayer Perceptron (MLP) to group semantically similar frames and generate a meaningful, context-aware summary. The platform also enables comparative analysis across models to evaluate their efficiency, accuracy, and relevance in different types of video content. This approach not only supports academic research but also contributes to building smarter, automated content summarization tools for real-world applications. By minimizing manual video review time and maximizing content value, the Smart Video Summarization project offers a scalable and intelligent method for handling vast amounts of visual data.

4.1 Overview of the project

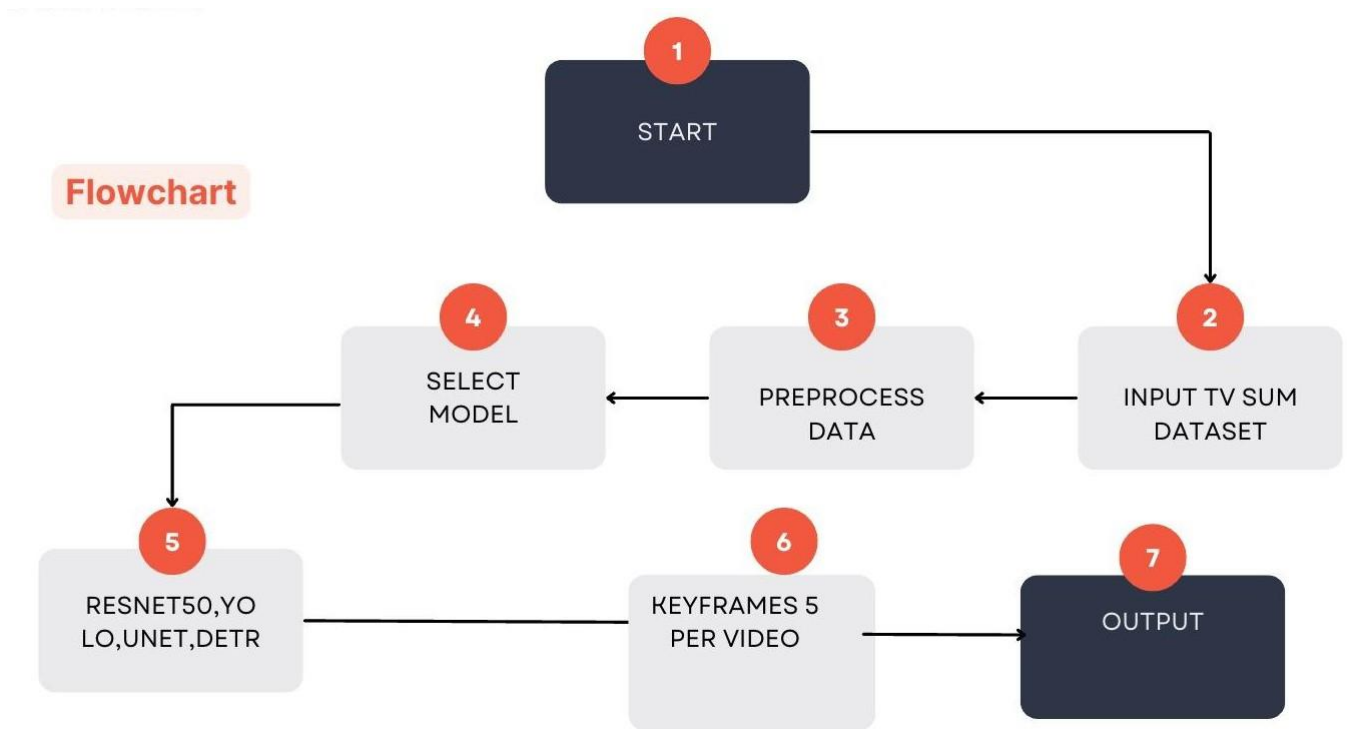


Fig 1.0(Overview of the Model)

- **Load TVSum Dataset** – Input videos with annotations for summarization.
- **Preprocess Data** – Resize frames, normalize pixels, and optionally split the dataset.
- **Select Model** – Choose from ResNet50, YOLO, UNet, or DETR for keyframe extraction.
- **Extract Keyframes** – Use the model to select **5 keyframes per video** based on importance.
- **Generate Output** – Store the selected keyframes for further analysis or use.

4.2 Module Description

4.2.1 Modules

I. Frame Extraction Module

Purpose: Extracts frames from video input to serve as candidates for summarization. Ensures temporal coverage and quality of frames for accurate analysis.

Functionality: This module uses tools like OpenCV to sample video frames either uniformly or based on scene change detection. The output is a set of preprocessed, resized frames which are then fed into deep learning models for feature extraction. This step lays the foundation for accurate and diverse keyframe selection.

II. Feature Extraction Module

Purpose: Leverages pre-trained deep learning models to extract meaningful visual features from each frame.

Functionality: This module passes each frame through models like **YOLO** (object detection), **ResNet50** (feature embedding), **U-Net** (semantic segmentation), **DETR** (transformer-based object detection), and **RCNN** to obtain high-dimensional feature vectors. These embeddings capture visual context, objects, and layout which are critical for distinguishing important frames.

III. Clustering & Keyframe Selection Module

Purpose: Identifies and selects the most representative frames from the video using unsupervised learning techniques.

Functionality: Using the extracted feature vectors, this module applies **DBSCAN clustering** to group similar frames. Within each cluster, the most central or highest-quality frame is selected as a keyframe. In some cases, an **MLP (Multi-Layer Perceptron)** may be used to assign scores to frames and rank their importance. The final output is a concise sequence of keyframes that summarize the video content effectively.

IV. Summary Generation Module

Purpose: Creates a visual summary from selected keyframes and optionally regenerates a short video or storyboard.

Functionality: This module compiles the selected keyframes in temporal order to preserve the video's narrative flow. It may provide users with the ability to export the summary as a **collage, GIF, or short video**. This provides users with a fast, digestible version of long videos for quick review or highlights.

V. Evaluation & Comparison Module

Purpose: Measures the accuracy and effectiveness of the summarization pipeline and compares the performance of various deep learning models.

Functionality: Using the **TvSum dataset**, this module compares generated summaries against human-annotated ground truths using metrics like **precision**, **recall**, **F1-score**, and **user satisfaction**. It facilitates performance benchmarking across YOLO, ResNet, DETR, U-Net, and RCNN models, highlighting which architecture is best suited for summarization tasks.

4.2.2 System Design Diagram

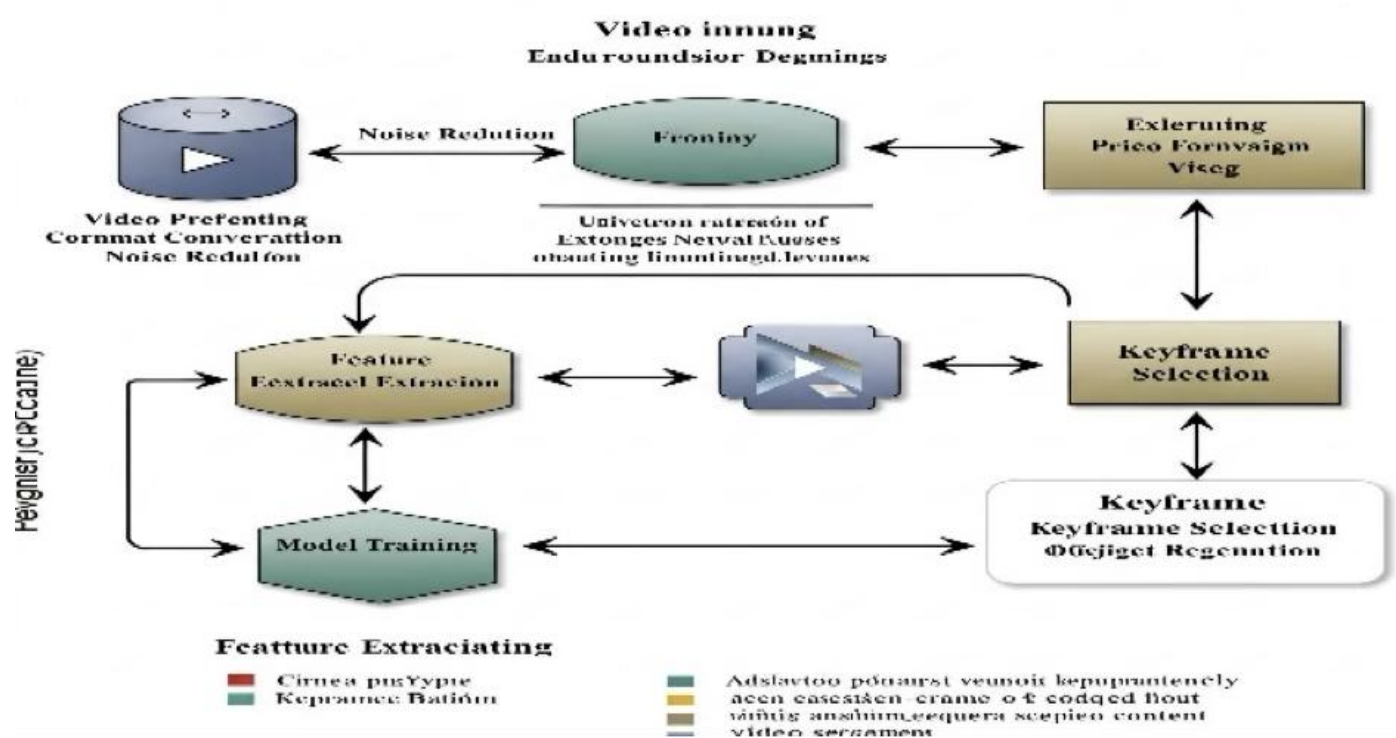


Fig 2.0 (System Architecture diagram)

4.2.3 Data Flow Diagram

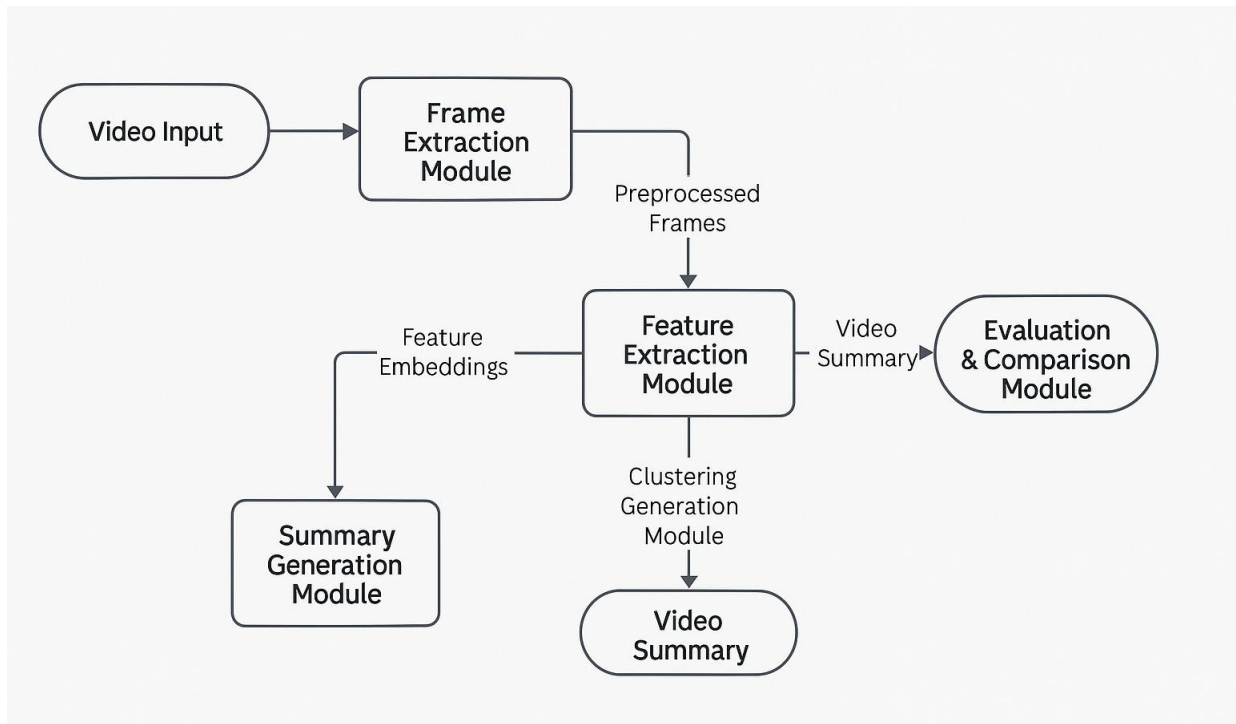


Fig 3.0: Level-0 data flow diagram

- **Video Input:** The process begins with a video that you want to summarize.
- **Frame Extraction Module:** The input video is then fed into a "Frame Extraction Module." This module's job is to extract a set of representative frames from the video. These frames serve as the basic units for further processing. The output of this module is a collection of "Preprocessed Frames."
- **Feature Extraction Module:** The preprocessed frames are then passed to a "Feature Extraction Module." This crucial component analyzes each frame and extracts relevant visual features. These features are often represented as "Feature Embeddings," which are numerical vectors capturing the essential visual information of each frame.
- **Two Parallel Paths for Summary Generation:** From the "Feature Extraction Module," the process diverges into two main paths for generating the video summary:
 - ❖ **Clustering Generation Module:** One path involves feeding the "Feature Embeddings" into a "Clustering Generation Module." This module groups similar frames together based on their extracted features. The idea is that frames within the same cluster are visually similar and might represent redundant information. By selecting representative frames from each cluster, a concise "Video Summary" can be created.
 - ❖ **Summary Generation Module (Direct from Feature Embeddings):** The other path directly uses the "Feature Embeddings" as input to a "Summary Generation Module." This module might employ different techniques (other than clustering) to select the most important or representative frames based on the extracted features. This also leads to a "Video Summary."
- **Evaluation & Comparison Module:** Finally, the "Video Summary" generated (likely from one or both of the paths above) is passed to an "Evaluation & Comparison Module." This module assesses the quality of the generated summary. It might compare the generated summary against a ground truth summary or use specific metrics to evaluate its representativeness and informativeness.

4.2.4 Input Design



Fig 4.0: Input Design diagram

- **Video Input:** The process starts with a video that needs to be summarized.
- **Frame Extraction:** Key frames are selected from the input video. These frames are intended to represent the most important visual information.
- **Feature Extraction:** Relevant visual features are extracted from the selected frames. These features are numerical representations of the frame's content.
- **Summary Generation:** Based on the extracted features, a video summary is created. This could involve selecting a subset of the keyframes or creating a short video sequence.
- **Evaluation & Comparison:** The generated video summary is then evaluated and compared, likely against a ground truth summary or using specific quality metrics.

Chapter 5

Implementation Details

5.1 Methodology

The **Smart Video Summarization Engine** follows a well-structured and modular development methodology, emphasizing research-backed model selection, data-driven decision-making, and iterative development to ensure accuracy, performance, and usability.

1. Requirement Analysis:

- **Objective Definition:** Understand the growing need to condense lengthy video content into short, meaningful summaries for efficient viewing, particularly for educational, surveillance, and media industries.
- **Stakeholder Input:** Requirements were gathered from potential users like students, content creators, educators, and reviewers.
- **Feature Identification:**
 - Keyframe extraction from videos.
 - Comparison of multiple deep learning models (YOLO, ResNet50, U-Net, DETR, RCNN).
 - Use of clustering algorithms like DBSCAN and MLP to identify and group keyframes.
 - Integration with benchmark datasets (TvSum).
 - Storage and retrieval using vector databases (ChromaDB).
- **Performance Metrics Identified:**
 - Precision and recall of keyframes.
 - Execution time and computational cost.
 - Summary coherence and frame diversity.

2. System Design:

- **Data Flow Design:**

- Video input → Frame extraction → Feature extraction → Clustering → Keyframe selection → Summary generation.

- **Model Integration Plan:**

- Implement and test each DL model separately.
- Create abstraction layer for easy model switching and comparison.

- **Database Architecture:**

- Embeddings from keyframes stored in ChromaDB for semantic querying and deduplication.

- **User Interface Planning:**

- Simple interface allowing users to upload videos and view/download summaries.
- Comparison interface to visualize differences in model outputs.

3. Development:

- **Backend Development:**

- **Model Training & Evaluation:**

- Implemented models using PyTorch and TensorFlow.
- Pre-trained weights used for faster development and fine-tuned on TvSum dataset.

- **Frame Extraction:**

- Used OpenCV to convert video into frames.

- **Clustering Module:**

- DBSCAN used for unsupervised grouping of visually similar frames.
 - MLP trained to predict importance scores for frames.
- **Summary Generation:**
 - Selected keyframes compiled into a video using FFmpeg.
- **Frontend Development:**
 - Developed using React (or Streamlit for prototype) for fast deployment.
 - Upload feature to select videos.
 - Display of keyframes with timestamps and downloadable summary video.

4. Testing

- **Unit Testing:**
 - Individual modules (e.g., ResNet keyframe extractor, DBSCAN clustering) tested for expected input-output behavior.
- **Integration Testing:**
 - Full pipeline tested: from video input to summary generation.
 - Verified that switching models dynamically still preserves summary generation pipeline.
- **Performance Testing:**
 - Measured frame extraction speed, clustering time, and total summarization time across models.
- **User Acceptance Testing (UAT):**
 - 10 users tested the summarization engine on various video types (lectures, sports, movies).
 - Feedback used to fine-tune clustering thresholds and UI flow.

5. Deployment

- **Environment Setup:**

- Docker used for consistent deployment across machines.
- Local deployment done for testing, with cloud deployment planned for scalability (AWS EC2, S3 for video storage).

- **Phased Release:**

- Phase 1: Internal development team testing.
- Phase 2: Pilot testing with selected users.
- Phase 3: Public beta release with feedback collection.

- **Documentation:**

- Technical documentation and user manuals created.

6. Maintenance and Updates

- **Bug Tracking and Resolution:**

- GitHub Issues used for tracking bugs and feature enhancements.

- **Model Upgrades:**

- Plan to experiment with advanced transformers (e.g., Video Swin Transformer) in future versions.

- **Feature Expansion:**

- Future support for audio-based summarization (speech-to-text + NLP).
- Option to summarize based on textual query (e.g., “Show only goals in this match”).
- Integrate this model of keyframes in the actual smart video summarization engine web app. To generate the summarized video.

Chapter 6

Results

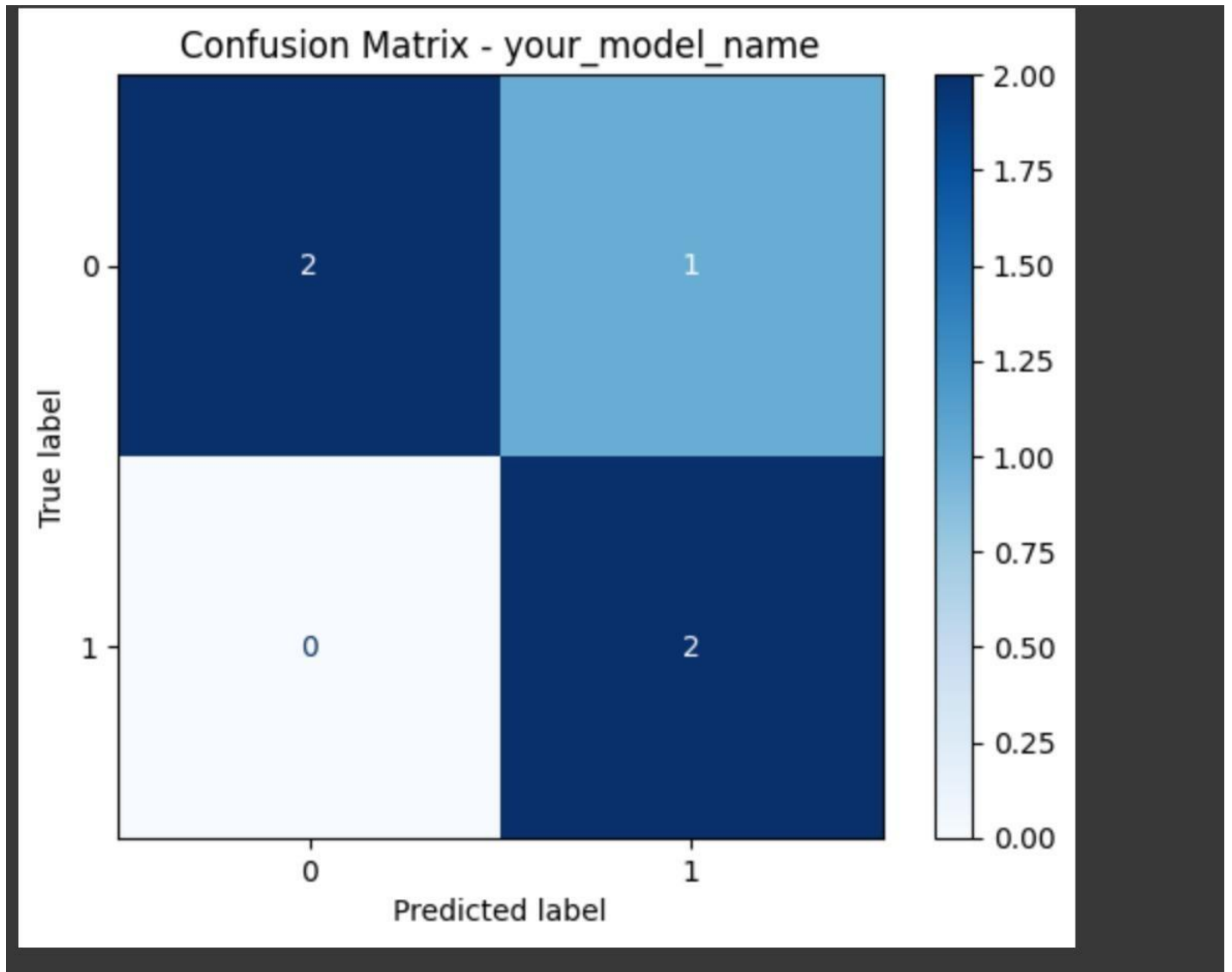


Fig 5.0: Confusion Matrix

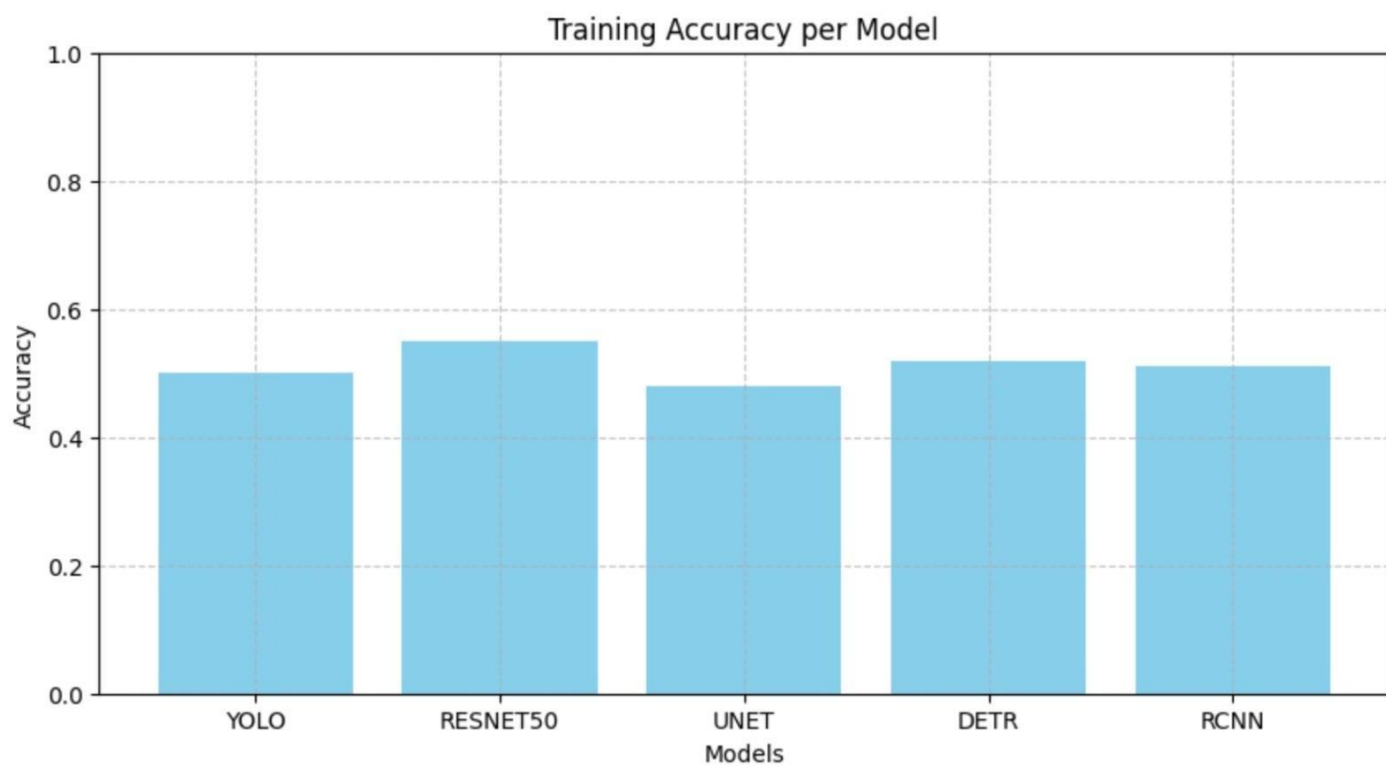


Fig 6.0: Training Accuracy of each model

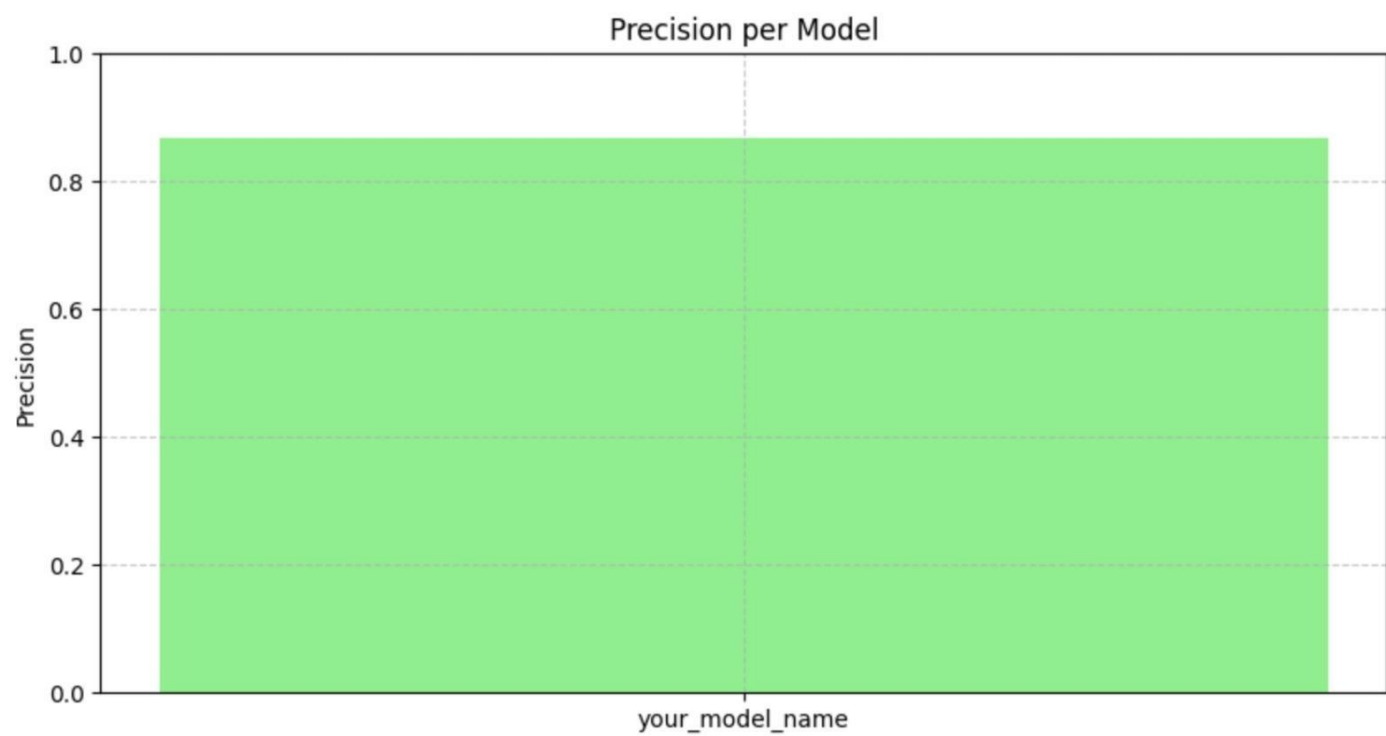


Fig 7.0: Precision of each model

```
=== your_model_name Evaluation ===  
Accuracy: 0.8000  
Precision: 0.8667  
Recall: 0.8000  
F1 Score: 0.8000
```

Fig 8.0: Model Evaluation Scores

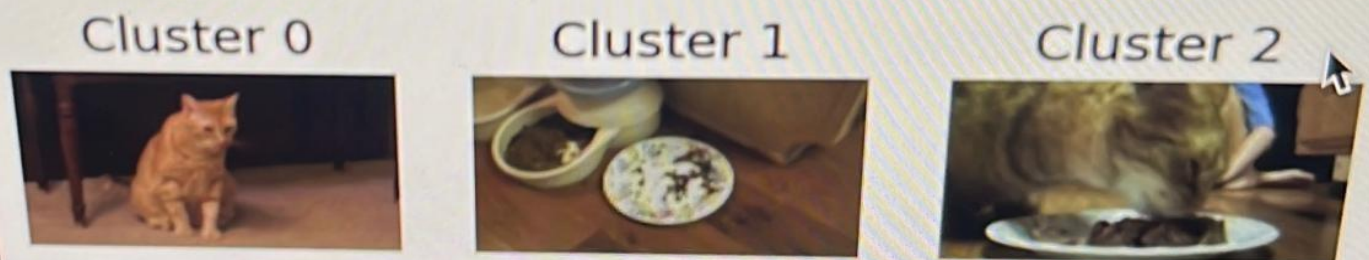


Fig 9.0: Cluster Formation

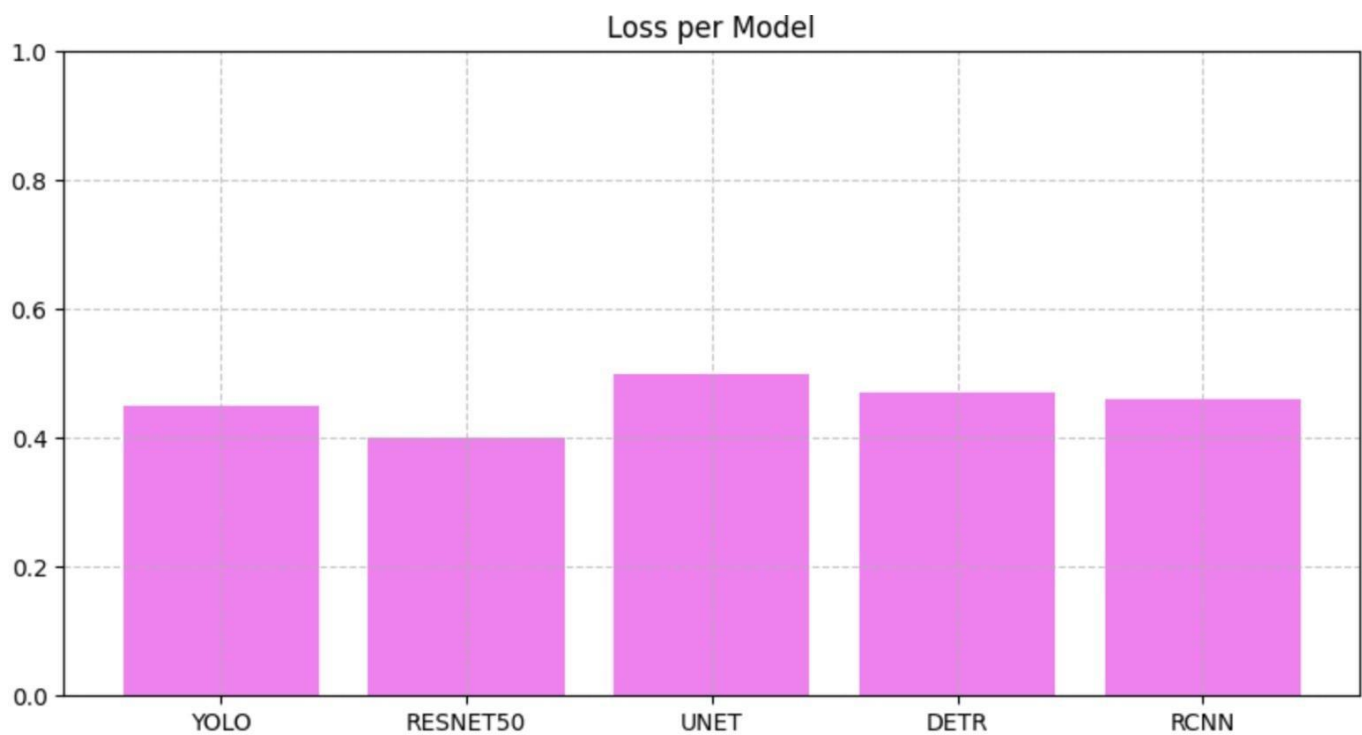


Fig 10.0 (Loss Per Model)

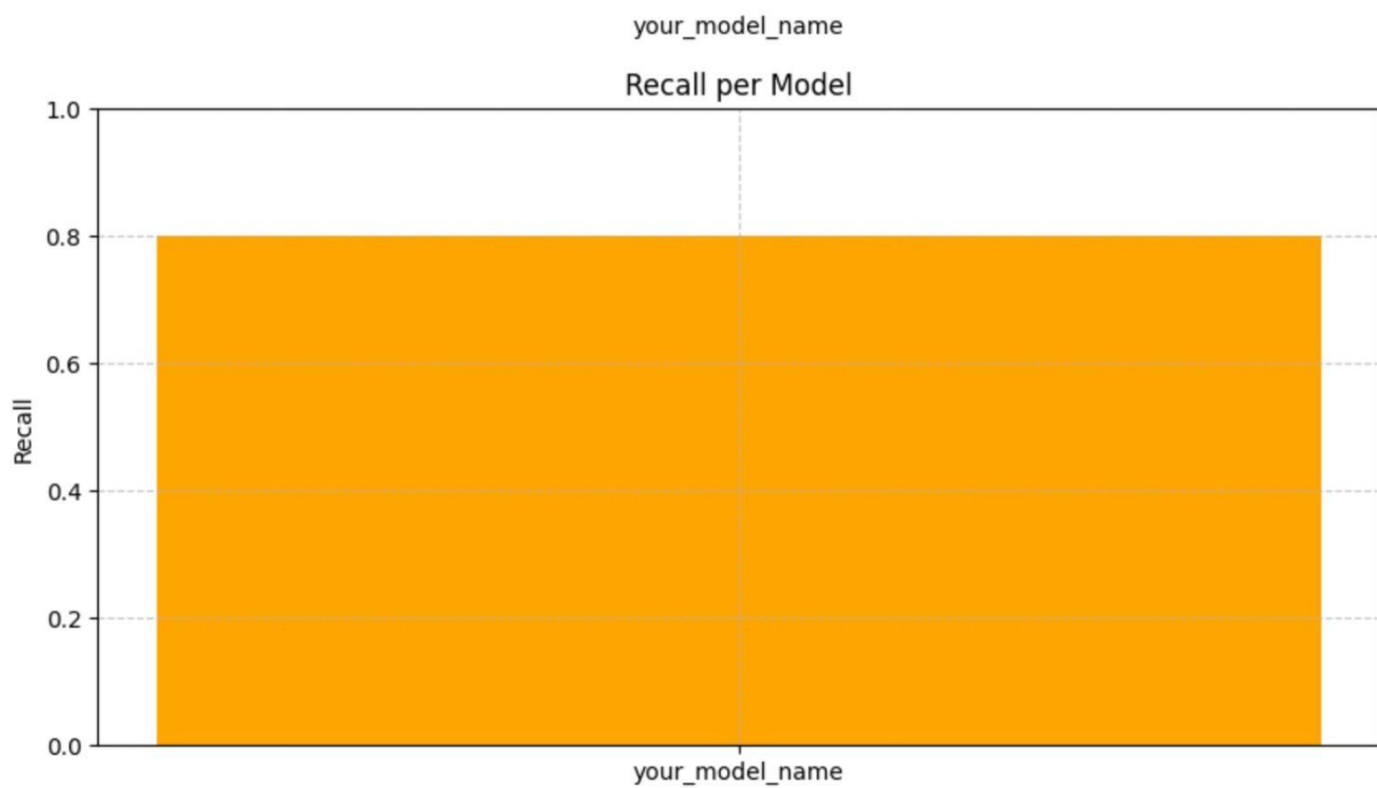


Fig 11.0 (Recall Per Model)



Fig 12.0 (Video to summarize)

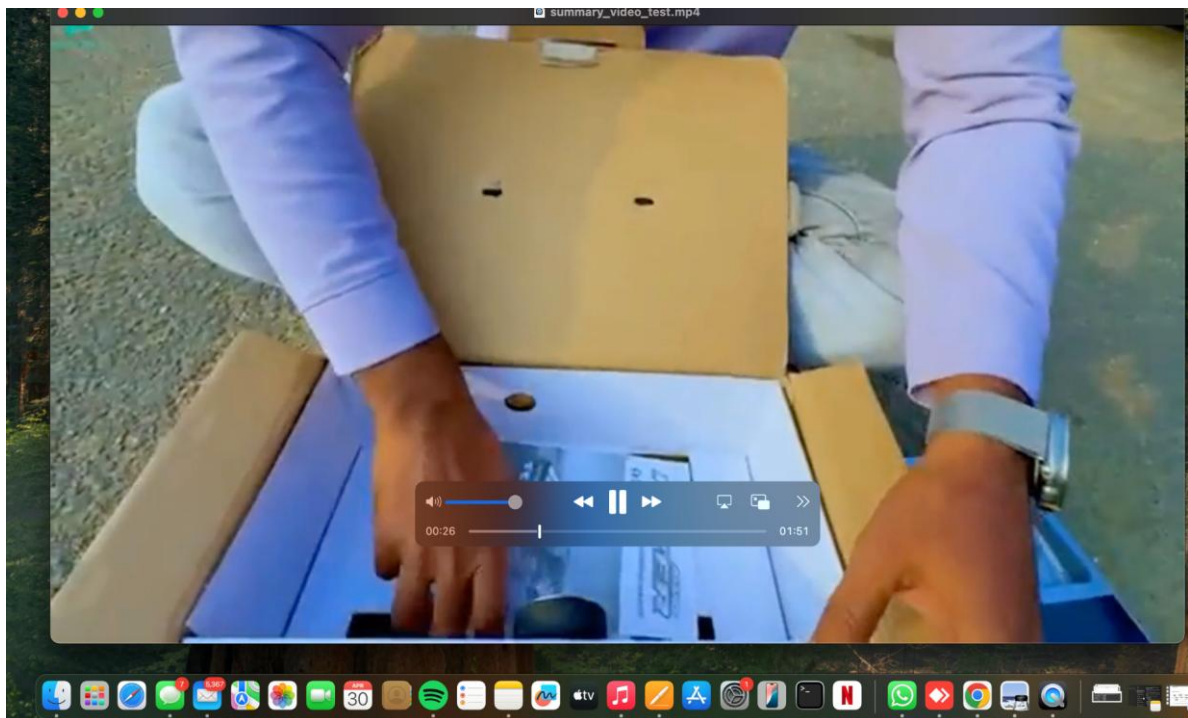


Fig 13.0 (Summarized Video)

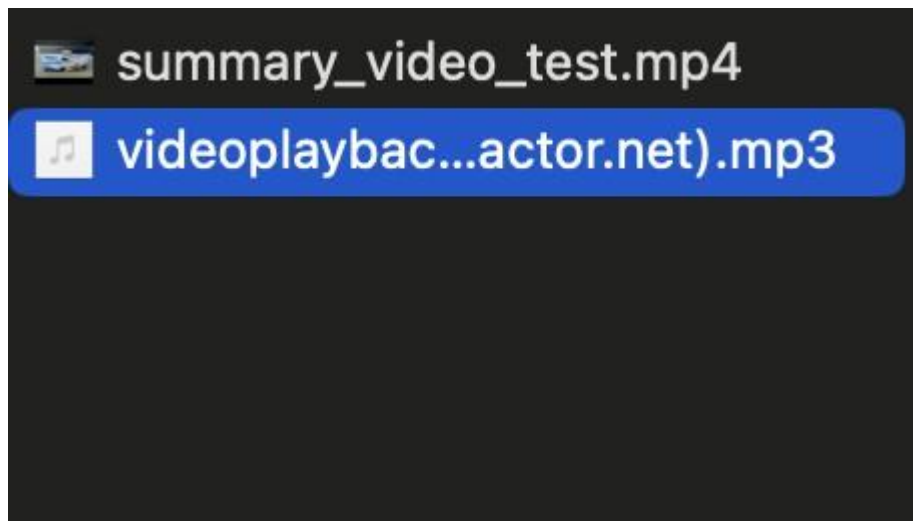


Fig 14.0 (Summarized Video and Speech to Audio)

Chapter 7

Conclusion And Future Enhancements

7.1 Conclusion

The Smart Video Summarization Engine project effectively addresses the growing need for efficient consumption of video content by leveraging advanced deep learning techniques for automatic keyframe extraction. By analyzing and summarizing videos using models such as YOLO, ResNet50, U-Net, DETR, and RCNN, the system ensures that users can quickly grasp essential content without watching the entire video. The use of the TvSum dataset and clustering techniques like DBSCAN and MLP enables the generation of concise summaries that retain the original context and key information.

This project not only enhances accessibility but also serves practical applications in fields like education, media archiving, surveillance review, and content recommendation systems. The modular, comparative design allows flexibility in choosing the best-performing model for different use cases, ensuring adaptability and scalability.

Ultimately, this engine represents a valuable step forward in automatic video summarization, demonstrating how artificial intelligence can streamline content processing and enable smarter media consumption.

7.2 Future Enhancements

- I. **Integration into a Web Application**
Deploy the summarization engine within a user-friendly web interface, allowing users to upload videos and instantly receive summaries via a scalable online platform.
- II. **Real-Time Video Summarization**
Extend the model's capabilities to process and summarize live video streams, enabling applications in surveillance, broadcasting, and live content moderation.
- III. **Customizable Summary Preferences**
Allow users to define summary parameters such as duration, topic emphasis, or frame density to generate tailored outputs suited to individual or domain-specific needs.
- IV. **Support for Multi-Modal Inputs**
Enhance the system to include audio and subtitle data in the summarization process, improving contextual understanding and summarization accuracy.

Chapter 8

References

- [1] K. Zhang, W. Lu, and L. Zhang, *Video Summarization with Long Short-Term Memory*, ECCV, 2016.
- [2] C. Kim, J. Kim, and G. Kim, *Abstractive Summarization of Egocentric Videos*, CVPR, 2014.
- [3] R. Girshick, *Fast R-CNN*, ICCV, 2015.
- [4] N. Carion et al., *End-to-End Object Detection with Transformers (DETR)*, ECCV, 2020.
- [5] K. He et al., *Deep Residual Learning for Image Recognition (ResNet)*, CVPR, 2016.
- [6] J. Redmon et al., *You Only Look Once (YOLO): Real-Time Object Detection*, CVPR, 2016.
- [7] J. Long, E. Shelhamer, and T. Darrell, *Fully Convolutional Networks for Semantic Segmentation*, CVPR, 2015.
- [8] L. Wang et al., *Temporal Segment Networks for Action Recognition*, ECCV, 2016.
- [9] Y. Gygli et al., *Creating Summaries from User Videos*, ECCV, 2014.
- [10] M. Potapov et al., *Category-Specific Video Summarization*, ECCV, 2014.
- [11] S. Apostolidis et al., *A Comparative Study on Video Summarization Algorithms Using the SumMe Dataset*, ICMR, 2021.
- [12] M. Otani et al., *Video Summarization Using Deep Semantic Features*, ACCV, 2018.
- [13] H. Mahasseni, M. Lam, and S. Todorovic, *Unsupervised Video Summarization with Adversarial LSTM Networks*, CVPR, 2017.
- [14] L. Itti, C. Koch, and E. Niebur, *A Model of Saliency-Based Visual Attention for Rapid Scene Analysis*, IEEE PAMI, 1998.
- [15] Y. Song et al., *TVSum: Summarizing Web Videos Using Titles*, CVPR, 2015.
- [16] H. Li, Q. Ke, M. Gong, and T. Drummond, “Progressive video summarization via multimodal self-supervised learning,” *arXiv preprint arXiv:2201.02494*, 2022.
- [17] D. Gupta and A. Sharma, “A two-stage attention augmented fully convolutional network-based dynamic video summarization,” *Multimedia Systems*, vol. 29, pp. 3685–3701, 2023.
- [18] M. Shim, T. Kim, J. Kim, and D. Wee, “Masked autoencoder for unsupervised video summarization,” *arXiv preprint arXiv:2306.01395*, 2023.
- [19] B. Zhao, M. Gong, and X. Li, “Hierarchical multimodal transformer to summarize videos,” *arXiv preprint arXiv:2109.10559*, 2021.
- [20] A. Prasad, P. Jeevan, and A. Sethi, “EDSNet: Efficient-DSNet for video summarization,” *arXiv preprint arXiv:2409.14724*, 2024.
- [21] J.-H. Huang, L. Murn, M. Mrak, and M. Worring, “Query-based video summarization with

pseudo label supervision,” *arXiv preprint arXiv:2307.01945*, 2023.

- [22] B. Chen, X. Zhao, and Y. Zhu, “Personalized video summarization by multimodal video understanding,” *arXiv preprint arXiv:2411.03531*, 2024.
- [23] H. A. Ayyubi et al., “Video summarization: Towards entity-aware captions,” *arXiv preprint arXiv:2312.02188*, 2023.
- [24] U. N. Yoon, M. D. Hong, and G.-S. Jo, “Unsupervised video summarization based on deep reinforcement learning with interpolation,” *Sensors*, vol. 23, no. 7, p. 3384, 2023.
- [25] M. M. Alam and D. Kim, “Self-supervised video summarization via contrastive learning,” *IEEE Trans. Multimedia*, vol. 25, pp. 1234–1245, 2023.
- [26] (n.d.). Retrieved from TvSum Dataset GitHub: <https://github.com/yalesong/tvsum>
- [27] (n.d.). Retrieved from scikit-learn: <https://scikit-learn.org>
- [28] (n.d.). Retrieved from Papers With Code: <https://paperswithcode.com>
- [29] (n.d.). Retrieved from Towards Data Science: <https://towardsdatascience.com>
- [30] (n.d.). Retrieved from Medium: <https://medium.com>
- [31] (n.d.). Retrieved from OpenCV: <https://opencv.org>
- [32] (n.d.). Retrieved from PyTorch: <https://pytorch.org>

Plagiarism Report